

CRASH COURSE PLAN

NestJS Crash Course

Lộ trình học chi tiết từng phần — kèm giải thích cơ chế hoạt động bên trong

Repo: [crash-course](#) / [nestjs-crash-course](#)

Mục lục

00 Kiến trúc tổng thể & Request Lifecycle

01 Khởi tạo & cấu trúc dự án

02 Controllers & Routing

03 Providers, Services & Dependency Injection

04 Modules

05 DTO & Validation

06 Pipes & Exception Filters

07 Middleware, Guards, Interceptors

08 Kết nối Database với TypeORM

09 Quan hệ dữ liệu (Relations)

10 Authentication (JWT + Passport)

11 Testing (Jest)

Cách dùng tài liệu này: Bạn tự code 100% từng phần. Mỗi phần có mục "Việc cần làm" (task) — không có code giải sẵn để copy, chỉ có ví dụ cú pháp minh họa để hiểu cách viết. Khi xong một phần, quay lại chat để mình review và giải thích tiếp phần sau.

Kiến trúc tổng thể & Request Lifecycle

Đây là phần quan trọng nhất để hiểu "mọi thứ hoạt động thế nào" trước khi đi vào từng phần nhỏ. NestJS không phải một framework hoàn toàn mới — nó là một lớp kiến trúc (architecture layer) xây trên nền HTTP server có sẵn (mặc định là **Express**, có thể đổi sang **Fastify**). NestJS lấy cảm hứng từ Angular: dùng **TypeScript decorators**, **Dependency Injection (DI)**, và tổ chức code theo **Module**.

1. Decorator & Reflection Metadata — "phép màu" đằng sau NestJS

Toàn bộ NestJS hoạt động dựa trên một cơ chế của TypeScript gọi là **decorator** kết hợp thư viện `reflect-metadata`. Khi bạn viết:

```
@Controller('posts')
export class PostsController { }
```

Decorator `@Controller('posts')` thực chất là một hàm chạy tại thời điểm class được định nghĩa (compile/load time). Nó không "chạy logic nghiệp vụ" — nó chỉ **gắn metadata** (dữ liệu mô tả) vào class đó, kiểu như dán nhãn: "class này là một Controller, prefix route là `/posts`". NestJS lưu metadata này bằng `Reflect.defineMetadata()`.

Khi ứng dụng khởi động (`bootstrap()`), NestJS quét toàn bộ metadata đã được gắn (dùng `Reflect.getMetadata()`) để biết: class nào là Controller, class nào là Provider, route nào ứng với method nào, cần inject dependency gì vào constructor... Đây chính là lý do NestJS gọi là "declarative" — bạn *khai báo* ý định bằng decorator, framework tự đọc và ráp nối lại.

2. Module Graph & Dependency Injection Container (IoC Container)

Khi app khởi động, NestJS xây dựng một **cây module** (module tree) bắt đầu từ `AppModule` (root module). Với mỗi module, nó đọc metadata trong `@Module({...})`:

- `imports` : các module khác mà module này cần dùng
- `controllers` : các controller thuộc module này
- `providers` : các service/class sẽ được đăng ký vào DI container
- `exports` : các provider mà module này "cho phép" module khác dùng lại

NestJS tạo ra một **IoC Container** (Inversion of Control) — một "kho chứa instance". Mỗi provider (mặc định là **singleton**) chỉ được khởi tạo **một lần duy nhất** trong toàn bộ vòng đời app. Khi class A cần class B trong constructor:

```
@Injectable()
export class PostsController {
  constructor(private readonly postsService: PostsService) {}
}
```

NestJS nhìn vào **type** của tham số constructor (nhờ TypeScript compile ra metadata kiểu dữ liệu), tra trong container xem đã có instance của `PostsService` chưa — nếu có thì tái sử dụng, nếu chưa thì tạo mới rồi lưu lại. Đây gọi là **constructor-based injection**. Bạn không bao giờ tự viết `new PostsService()` — NestJS làm việc đó cho bạn. Lợi ích: dễ test (mock được), dễ thay thế implementation, giảm coupling giữa các class.

Encapsulation theo module: Provider trong module A *không* tự động dùng được ở module B — trừ khi A `export` nó ra và B `import` module A vào. Đây là ranh giới đóng gói (giống package-private), giúp kiến trúc lớn không bị rối.

3. Request Lifecycle — thứ tự một request đi qua hệ thống

Đây là sơ đồ quan trọng nhất cần thuộc lòng. Khi một HTTP request đến, nó đi qua các lớp theo đúng thứ tự sau:



Hiểu sơ đồ này sẽ giải thích được rất nhiều câu hỏi "tại sao": tại sao Guard chạy trước khi vào được Controller (để chặn truy cập trái phép sớm), tại sao Pipe luôn chạy trước khi handler nhận được data (để

đảm bảo data đã sạch/hợp lệ), tại sao Interceptor có thể "bọc" (wrap) cả input lẫn output (vì nó chạy cả trước và sau).

4. Provider Scope (phạm vi sống của instance)

Scope	Mặc định?	Giải thích
DEFAULT (Singleton)	✓ Có	Một instance duy nhất, dùng chung cho toàn bộ app suốt vòng đời server
REQUEST	Không	Tạo instance mới cho mỗi request — dùng khi cần lưu state riêng theo request (vd: multi-tenant)
TRANSIENT	Không	Tạo instance mới mỗi lần được inject — hiếm dùng

99% trường hợp trong crash course này bạn sẽ dùng Singleton (mặc định) — không cần khai báo gì thêm.

Khởi tạo & cấu trúc dự án

Mục tiêu: Hiểu Nest CLI tạo ra những gì, và luồng khởi động (bootstrap) của một app NestJS.

1.1 Cài đặt & tạo project

```
npm i -g @nestjs/cli
nest new nestjs-crash-course
```

Nest CLI dùng schematics (khuôn mẫu code) để sinh ra cấu trúc chuẩn, tương tự `create-react-app` hay `vue create`.

1.2 Cấu trúc thư mục mặc định

File	Vai trò
<code>src/main.ts</code>	Entry point — nơi app thực sự khởi động
<code>src/app.module.ts</code>	Root Module — gốc của cây module
<code>src/app.controller.ts</code>	Controller mẫu, xử lý route <code>/</code>
<code>src/app.service.ts</code>	Service mẫu, chứa logic nghiệp vụ
<code>nest-cli.json</code>	Cấu hình Nest CLI (build, schematics...)
<code>tsconfig.json</code>	Cấu hình TypeScript — chú ý <code>experimentalDecorators</code> và <code>emitDecoratorMetadata</code> phải bật (đây là 2 cờ TS bắt buộc để decorator + DI hoạt động)

1.3 Giải phẫu main.ts

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(process.env.PORT ?? 3000);
}
bootstrap();
```

`NestFactory.create(AppModule)` chính là bước NestJS đọc `AppModule`, duyệt toàn bộ `imports` để quy, đăng ký hết Controllers/Providers vào IoC container, resolve toàn bộ dependency injection — tất cả xảy ra **trước** khi `listen()` được gọi. Nếu có lỗi thiếu dependency hoặc circular dependency, app sẽ crash ngay tại bước `create()` này (rất hữu ích để debug).

Việc cần làm:

1. Tạo project bằng `nest new nestjs-crash-course`
2. Chạy `npm run start:dev` , mở `http://localhost:3000` , xác nhận thấy "Hello World!"
3. Đọc kỹ 4 file: `main.ts`, `app.module.ts`, `app.controller.ts`, `app.service.ts` — thử tự vẽ lại sơ đồ ai gọi ai
4. Thử đổi cổng bằng biến môi trường `PORT=4000 npm run start:dev`

Controllers & Routing

Mục tiêu: Hiểu cách NestJS map một HTTP request tới đúng method trong Controller, và cách lấy dữ liệu từ request.

2.1 Cơ chế map route

`@Controller('posts')` đăng ký prefix `/posts`. Mỗi method bên trong gắn thêm decorator HTTP method:

```
@Controller('posts')
export class PostsController {
  @Get()          // GET /posts
  @Get(':id')     // GET /posts/:id
  @Post()         // POST /posts
  @Put(':id')     // PUT /posts/:id
  @Delete(':id')  // DELETE /posts/:id
}
```

Khi bootstrap, NestJS quét toàn bộ Controller đã đăng ký trong `controllers: []` của module, đọc metadata của từng method, rồi đăng ký route đó thẳng vào Express router bên dưới. Về bản chất, NestJS chỉ là một lớp "khai báo" đặt lên trên `app.get()`, `app.post()` của Express.

2.2 Param Decorators — lấy dữ liệu từ request

Decorator	Lấy gì	Ví dụ
<code>@Param('id')</code>	Route param	<code>/posts/5</code> → <code>id = "5"</code>
<code>@Query('search')</code>	Query string	<code>/posts?search=abc</code> → <code>"abc"</code>
<code>@Body()</code>	Toàn bộ request body (JSON)	dùng cho POST/PUT
<code>@Headers('authorization')</code>	HTTP header	lấy token
<code>@Req()</code>	Object request thô của Express	ít dùng, phá vỡ tính "framework-agnostic"

Các decorator này thực chất cũng chỉ là metadata — NestJS đọc metadata đó, biết phải lấy giá trị nào từ `req` gốc của Express, rồi **tự động truyền đúng thứ tự** vào tham số hàm của bạn khi gọi handler. Đây là lý do bạn không cần viết `req.params.id` thủ công như Express thuần.

Việc cần làm:

1. Tạo `PostsController` bằng `nest g controller posts`

2. Viết 5 route CRUD cơ bản (GET all, GET one, POST, PUT, DELETE) — trả về mảng cứng trong bộ nhớ (chưa cần service/database)
3. Test bằng Postman/curl/Thunder Client cho từng route

Providers, Services & Dependency Injection

Mục tiêu: Tách logic nghiệp vụ ra khỏi Controller, hiểu sâu cách DI container resolve dependency.

3.1 Tại sao tách Service?

Nguyên tắc **Separation of Concerns**: Controller chỉ nên lo việc "nhận request → gọi service → trả response". Toàn bộ logic nghiệp vụ (tính toán, thao tác dữ liệu, gọi database) nằm trong Service. Điều này giúp: Controller dễ đọc, Service tái sử dụng được ở nơi khác (vd: một cron job cũng gọi lại service đó), và quan trọng nhất — **dễ test** (test Service không cần giả lập HTTP request).

3.2 @Injectable() nghĩa là gì?

```
@Injectable()
export class PostsService {
  private posts = [ ... ];
  findAll() { return this.posts; }
}
```

`@Injectable()` đánh dấu class này **có thể được đưa vào DI container** để inject cho class khác. Bản thân từ "Injectable" nghĩa là "có thể tiêm vào" — không phải class này tự tiêm, mà là nó *đủ điều kiện để bị tiêm vào constructor của class khác*.

3.3 Luồng resolve dependency chi tiết

1. App bootstrap → NestJS đọc `@Module({ providers: [PostsService] })`
2. NestJS thấy `PostsController` có constructor(`private postsService: PostsService`)
3. NestJS tra IoC container: "đã có instance `PostsService` chưa?"
 - Chưa có → tạo mới `new PostsService()` → LƯU VÀO container (singleton)
 - Đã có → lấy lại instance cũ, KHÔNG tạo mới
4. NestJS gán instance đó vào `this.postsService` của `PostsController`
5. Toàn bộ app dùng CHUNG 1 instance `PostsService` này

Vì sao phải khai báo providers trong @Module? Nếu bạn quên thêm `PostsService` vào mảng `providers` của module, NestJS sẽ báo lỗi `Nest can't resolve dependencies` ngay lúc khởi động — vì nó không biết class đó tồn tại trong container.

Việc cần làm:

1. Tạo `PostsService` bằng `nest g service posts`
2. Chuyển toàn bộ logic (mảng posts, các hàm CRUD) từ Controller sang Service
3. Inject Service vào Controller qua constructor, Controller chỉ gọi lại các method của Service

Modules

Mục tiêu: Tổ chức app lớn theo feature module thay vì nhồi hết vào AppModule.

4.1 Feature Module là gì?

Khi app lớn dần, bạn không nhét tất cả Controller/Service vào `AppModule`. Thay vào đó, mỗi "tính năng" (feature) có module riêng — vd `PostsModule`, `UsersModule`, `AuthModule` — rồi `AppModule` chỉ `import` các module con này lại.

```
@Module({
  controllers: [PostsController],
  providers: [PostsService],
  exports: [PostsService], // cho module khác dùng lại PostsService
})
export class PostsModule {}
```

`nest g resource posts` là lệnh "tất cả trong một" — tự sinh Controller, Service, Module, DTO, Entity cùng lúc và tự động import vào AppModule. Rất hay dùng trong thực tế.

Việc cần làm:

1. Refactor `PostsController` / `PostsService` đã làm ở Part 2-3 vào một `PostsModule` riêng
2. Import `PostsModule` vào `AppModule`, xoá bớt khai báo trùng lặp ở AppModule
3. Thử tạo thêm 1 module mới bằng `nest g resource users` để quen tay với CLI

DTO & Validation

Mục tiêu: Đảm bảo dữ liệu client gửi lên luôn đúng định dạng trước khi vào tới Service.

5.1 DTO (Data Transfer Object) là gì?

DTO là một class mô tả "hình dạng" dữ liệu được truyền giữa client và server (khác với Entity — mô tả bảng trong database, sẽ học ở Part 8). Ví dụ `CreatePostDto` mô tả body hợp lệ khi tạo post mới.

5.2 class-validator + ValidationPipe hoạt động thế nào

```
export class CreatePostDto {
  @IsString()
  @IsNotEmpty()
  title: string;

  @IsString()
  body: string;
}
```

Decorator như `@IsString()` gắn metadata validation lên từng field (thư viện `class-validator`). Khi request tới, `ValidationPipe` (chạy ở bước "Pipes" trong request lifecycle — xem Part 0) sẽ:

1. Lấy raw JSON body
2. `class-transformer` biến raw object thành instance thật của class DTO (`plainToInstance`)
3. `class-validator` chạy tất cả rule decorator đã khai báo, thu thập lỗi
4. Nếu có lỗi → tự động throw `BadRequestException` (HTTP 400) kèm danh sách lỗi chi tiết, request **không bao giờ chạm tới Controller**
5. Nếu hợp lệ → object sạch được truyền vào `@Body()` của handler

Đăng ký global (áp dụng cho toàn bộ app) trong `main.ts` :

```
app.useGlobalPipes(new ValidationPipe({
  whitelist: true, // tự động loại bỏ field lạ không khai báo trong DTO
  forbidNonWhitelisted: true, // field lạ → throw lỗi thay vì âm thầm bỏ qua
}));
```

Việc cần làm:

1. Cài `class-validator` `class-transformer`
2. Tạo `CreatePostDto` và `UpdatePostDto` (dùng `PartialType` từ `@nestjs/mapped-types` để field đều optional)
3. Gắn `ValidationPipe` global trong `main.ts`

4. Test gửi body thiếu field / sai kiểu dữ liệu → xác nhận nhận về lỗi 400 rõ ràng

Pipes & Exception Filters

Mục tiêu: Tự viết pipe/filter riêng, hiểu cách NestJS xử lý lỗi tập trung.

6.1 Custom Pipe

Pipe là class implement interface `PipeTransform` với method `transform(value, metadata)` — nhận giá trị đầu vào, trả về giá trị đã biến đổi (hoặc throw lỗi nếu không hợp lệ). Pipe có sẵn hay dùng:

`ParseIntPipe` (ép `:id` từ string sang number), `ParseUUIDPipe`.

6.2 Exception hierarchy

NestJS có sẵn class `HttpException` và các class con tương ứng mã HTTP: `NotFoundException` (404), `BadRequestException` (400), `UnauthorizedException` (401), `ForbiddenException` (403)... Khi bạn `throw new NotFoundException('Post not found')` ở bất kỳ đâu (Service hay Controller), NestJS có sẵn một **Exception Filter mặc định** bắt lấy, format thành JSON response chuẩn và trả về client — bạn không cần try/catch thủ công trong từng route.

6.3 Custom Exception Filter

```
@Catch(HttpException)
export class HttpExceptionFilter implements ExceptionFilter {
  catch(exception: HttpException, host: ArgumentsHost) {
    // tùy biến format response lỗi, log lỗi, v.v.
  }
}
```

`@Catch(HttpException)` nghĩa là filter này chỉ bắt lỗi thuộc loại `HttpException` (và các class con). `ArgumentsHost` cho phép lấy lại request/response gốc để tự viết response lỗi theo format riêng của bạn (vd chuẩn hoá `{ statusCode, message, timestamp }`).

Việc cần làm:

1. Trong `PostsService`, khi không tìm thấy post theo id → throw `NotFoundException`
2. Viết 1 pipe tùy chỉnh đơn giản (vd validate id phải là số dương)
3. Viết 1 exception filter tùy chỉnh để format lại response lỗi theo ý bạn, đăng ký global bằng `app.useGlobalFilters()`

Middleware, Guards, Interceptors

Mục tiêu: Nắm 3 lớp còn lại trong request lifecycle, biết dùng đúng công cụ cho đúng việc.

7.1 Middleware — chạy sớm nhất, gần Express nhất

```
@Injectable()
export class LoggerMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: NextFunction) {
    console.log(`${req.method} ${req.url}`);
    next(); // bắt buộc gọi next() để đi tiếp, nếu không request sẽ treo
  }
}
```

Middleware KHÔNG khai báo trong `@Module({...})` như Controller/Provider — nó được gắn thủ công qua method `configure()` trong module, dùng `MiddlewareConsumer` để chỉ định áp dụng cho route nào.

7.2 Guards — trả lời câu hỏi "có được vào không?"

```
@Injectable()
export class AuthGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean {
    const request = context.switchToHttp().getRequest();
    return !!request.headers.authorization; // true = cho qua, false = 403
  }
}
```

Guard implement `CanActivate`, trả về `true / false` (hoặc `Promise<boolean>`). Đây là nơi chuẩn để đặt logic **Authorization** (đã đăng nhập chưa, có đúng role không) — khác với Middleware ở chỗ Guard có quyền truy cập `ExecutionContext` (biết được đang gọi tới Controller/handler nào, hữu ích để đọc thêm metadata custom qua `Reflector`, ví dụ decorator `@Roles('admin')` tự viết).

7.3 Interceptors — bọc quanh cả input lẫn output

```
@Injectable()
export class TransformInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    return next.handle().pipe(
      map(data => ({ data, success: true }))
    );
  }
}
```

Interceptor dùng **RxJS Observable**. `next.handle()` trả về Observable đại diện cho kết quả của route handler — bạn có thể `.pipe(map(...))` để biến đổi response trước khi trả về client (vd bọc response chuẩn `{ data, success }`), hoặc đo thời gian xử lý (log trước và sau `next.handle()`).

7.4 So sánh nhanh

Công cụ	Dùng khi nào	Chạy ở đâu trong lifecycle
Middleware	Logging, đọc/sửa raw request sớm	Sớm nhất, trước routing
Guard	Authorization (được phép truy cập không?)	Sau routing, trước Interceptor
Interceptor	Transform response, logging có ngữ cảnh handler, cache	Bọc quanh route handler
Pipe	Validate/transform input	Ngay trước khi vào handler
Exception Filter	Xử lý lỗi tập trung	Bất cứ khi nào có exception

Việc cần làm:

- Viết `LoggerMiddleware` log method + url + thời gian, áp dụng cho toàn bộ route
- Viết `AuthGuard` đơn giản kiểm tra header `Authorization` có tồn tại, áp lên route xóa/sửa post
- Viết `TransformInterceptor` bọc mọi response thành công theo format `{ statusCode, data }`

Kết nối Database với TypeORM

Mục tiêu: Thay mảng dữ liệu cứng trong bộ nhớ bằng database thật (SQLite để đơn giản, có thể nâng cấp Postgres sau).

8.1 TypeORM là gì

TypeORM là một **ORM** (Object-Relational Mapping) — cho phép thao tác database bằng object/class TypeScript thay vì viết SQL tay. NestJS tích hợp sẵn qua package `@nestjs/typeorm`.

8.2 Entity — class ánh xạ tới bảng

```
@Entity()
export class Post {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  title: string;

  @Column()
  body: string;
}
```

`@Entity()` đánh dấu class này ánh xạ tới một bảng. Mỗi `@Column()` là một cột. TypeORM đọc metadata này để tự sinh câu SQL `CREATE TABLE` (nếu bật `synchronize: true` — chỉ nên dùng khi dev, không dùng ở production vì có thể mất dữ liệu khi đổi schema).

8.3 Repository Pattern

```
constructor(
  @InjectRepository(Post)
  private postsRepository: Repository<Post>,
) {}

findAll() { return this.postsRepository.find(); }
findOne(id: number) { return this.postsRepository.findOneBy({ id }); }
create(dto: CreatePostDto) { return this.postsRepository.save(dto); }
```

`@InjectRepository(Post)` là một biến thể đặc biệt của DI — thay vì inject trực tiếp một class do bạn viết, bạn inject một **Repository object** mà TypeORM tự sinh ra cho Entity `Post`. Repository này có sẵn các method `find`, `findOneBy`, `save`, `delete` ... — về bản chất đây vẫn là DI container, chỉ là "provider" ở đây được đăng ký bởi `TypeOrmModule.forFeature([Post])` thay vì bạn tự viết `@Injectable()`.

Việc cần làm:

1. Cài `@nestjs/typeorm typeorm sqlite3`
2. Cấu hình `TypeOrmModule.forRoot()` trong AppModule (type: sqlite, database file, synchronize: true khi dev)
3. Tạo `Post` entity thay cho mảng cứng
4. Đăng ký `TypeOrmModule.forFeature([Post])` trong PostsModule, inject Repository vào PostsService
5. Chuyển toàn bộ hàm CRUD trong service sang gọi Repository thay vì thao tác mảng

Quan hệ dữ liệu (Relations)

Mục tiêu: Thêm entity thứ 2 (Comment) và liên kết với Post — hiểu cách TypeORM quản lý khoá ngoại.

9.1 One-to-Many / Many-to-One

```
// Trong Post entity
@OneToMany(() => Comment, (comment) => comment.post)
comments: Comment[];

// Trong Comment entity
@ManyToOne(() => Post, (post) => post.comments)
post: Post;
```

Một **Post** có nhiều **Comment** (OneToMany), mỗi **Comment** thuộc về đúng một **Post** (ManyToOne) — hai decorator này luôn đi thành cặp, tham số thứ 2 là hàm trả ngược lại field ở phía bên kia để TypeORM biết đây là 2 đầu của cùng 1 quan hệ.

9.2 Eager vs Lazy loading

eager: true trong decorator quan hệ nghĩa là mỗi lần query Post, TypeORM tự động JOIN kèm luôn Comment. Nếu không bật, bạn phải chủ động yêu cầu bằng **relations: ['comments']** khi query — cách này kiểm soát hiệu năng tốt hơn (tránh JOIN thừa khi không cần).

Việc cần làm:

1. Tạo **Comment** entity + **CommentsModule** (dùng **nest g resource comments**)
2. Thiết lập quan hệ OneToMany/ManyToOne giữa Post và Comment
3. Viết route **GET /posts/:id** trả về post kèm danh sách comment của nó

Authentication (JWT + Passport)

Mục tiêu: Hiểu toàn bộ luồng đăng ký → đăng nhập → cấp token → bảo vệ route bằng token.

10.1 Luồng tổng quan

1. POST /auth/register
 - hash password bằng bcrypt (KHÔNG BAO GIỜ lưu plain text password)
 - lưu user vào database
2. POST /auth/login
 - tìm user theo email
 - so sánh password gửi lên với hash đã lưu (bcrypt.compare)
 - nếu khớp → tạo JWT token (chứa payload: userId, email...) ký bằng secret key
 - trả token về client
3. Client lưu token, gửi kèm mọi request sau đó:
Header: Authorization: Bearer
4. Route được bảo vệ gắn @UseGuards(AuthGuard('jwt'))
 - PassportStrategy tự động verify chữ ký token bằng secret key
 - nếu hợp lệ → giải mã payload, gắn vào request.user
 - nếu không hợp lệ/hết hạn → tự động trả 401 Unauthorized

10.2 Vì sao dùng Passport?

@nestjs/passport là lớp tích hợp giữa NestJS và thư viện Passport.js (chuẩn công nghiệp cho authentication ở Node.js). Bạn định nghĩa một JwtStrategy (kế thừa PassportStrategy) khai báo cách lấy token (từ header), secret key để verify, và hàm validate(payload) chạy sau khi token đã được xác minh hợp lệ — trả về gì trong validate() thì thứ đó sẽ được gán vào request.user.

10.3 Vì sao hash password bằng bcrypt

Bcrypt là thuật toán hash **một chiều** (không thể giải mã ngược) và có "salt" ngẫu nhiên tích hợp sẵn — cùng 1 password, hash ra 2 lần sẽ cho 2 chuỗi khác nhau, chống lại rainbow table attack. Khi login, không "giải mã" hash để so sánh — mà hash lại password vừa nhập rồi so sánh 2 chuỗi hash (bcrypt.compare làm việc này).

Việc cần làm:

1. Tạo User entity, AuthModule
2. Cài bcrypt, @nestjs/jwt, @nestjs/passport, passport-jwt
3. Viết route register (hash password trước khi lưu) và login (verify + trả JWT)

4. Viết `JwtStrategy` + `JwtAuthGuard` , áp dụng lên các route cần đăng nhập (vd tạo/xoá post phải đăng nhập)

Testing với Jest

Mục tiêu: Viết unit test cho Service mà không cần khởi động cả app.

11.1 Unit test Service

Nest CLI tích hợp sẵn Jest. Với unit test, bạn dùng `Test.createTestingModule()` để tạo một "module giả lập" chỉ chứa đúng provider cần test — có thể **mock** Repository/dependency khác thay vì dùng database thật, giúp test chạy nhanh và độc lập.

11.2 e2e test

e2e (end-to-end) test khởi động toàn bộ app thật (dùng `supertest` để gửi HTTP request thật tới app trong bộ nhớ), kiểm tra toàn bộ luồng từ HTTP request tới response — gần với hành vi thực tế nhất nhưng chạy chậm hơn unit test.

Việc cần làm (tuỳ chọn, làm sau khi xong Part 1-10):

1. Viết unit test cho `PostsService.findAll()` và `findOne()` với repository được mock
2. Viết 1 e2e test đơn giản cho route `GET /posts`